ENSAE Paris
Institut Polytechnique de Paris
Python Programming Project

MAXIMUM MATCHING PROBLEMS USING GRID REPRESENTATION.

Pierre Robin Schnepf
David Florigny

Supervisor:
Prof. Patrick Loiseau

ENSAE PYTHON PROJECT - academic year 2024/2025

Abstract

This report presents the final version of our project, which addresses a matching problem on grids using various algorithms. We describe our implementation details, data structures, and solver strategies (Greedy, Brute Force, Hungarian, Ford-Fulkerson). We also summarize the computational complexity of each algorithm, and provide empirical results showcasing their performance on representative grid instances.

Contents

1 Introduction

The purpose of this project is to compute an optimal matching on a 2D grid, where certain pairs of cells can be combined based on color constraints, adjacency, and cost functions. A cell can only be used once, and the aim is to minimize the total cost across all valid pairs (or account for unpaired cells if necessary). Typical applications include puzzle games, resource-allocation tasks, and general combinatorial optimization problems with grid-like constraints.

Our codebase includes a `Grid` class to handle loading and manipulation of grid data, a `Graph` abstraction (switching from adjacency matrices to adjacency lists), and various solver classes such as `SolverGreedy`, `SolverBruteForce`, `SolverHungarian`, and `SolverFordFulkerson`. We rely on unit tests to verify correctness and produce repeatable results.

2 Data Structures and Implementation

2.1 Grid Representation

We store the grid as follows:

- **Dimension:** $\texttt{n} \times \texttt{m}$.
- **Colors:** An integer matrix $\texttt{color}[i][j]$, where color codes indicate white, forbidden, etc.
- **Values:** An integer matrix $\texttt{value}[i][j]$. The absolute difference $|v_1 - v_2|$ is used as the cost of pairing two cells, or a large penalty otherwise.

We also provide methods `is_forbidden()`, `is_peace()`, and pair utilities like `all_pairs()` to list valid adjacency pairs.

2.2 Graph Representation

To handle maximum flow and bipartite matching logic, we use a **graph** class that stores:

- **n** = number of vertices,
- **adj_list** = a list of dictionaries, where `adj_list`$[u]$ maps neighbor vertex v to a weight/-capacity.

For bipartite matching, we create an `SSBipartiteGraph` class that adds a source and sink, and splits grid cells into two sets based on parity: $(i+j)$ even connect to the source, and $(i+j)$ odd connect to the sink. Internal edges connect adjacent cells that satisfy color constraints.

3 Solvers and Complexity

We implement four distinct approaches:

3.1 Brute Force

- **Method:** Enumerate all subsets of valid pairs. For each subset, check if no cell is used more than once. Compute the global cost and retain the best.
- **Complexity:** $\mathcal{O}(2^P)$, where P is the number of adjacent pairs in the grid. This becomes rapidly infeasible as the grid grows, but it guarantees an optimal solution for small instances.
- **Proof:**

  Let E be a finite set such that $\mathrm{Card}(E) = P$. We wish to prove that the number of subsets of E, denoted by $\mathcal{P}(E)$, is equal to 2^P.

  For every integer k between 0 and P, the number of subsets of E containing exactly k elements is given by the binomial coefficient

  $$\binom{P}{k}.$$

  Thus, the total number of subsets is

  $$\mathrm{Card}(\mathcal{P}(E)) = \sum_{k=0}^{P} \binom{P}{k}.$$

  According to the binomial theorem, we have

  $$(1+1)^P = \sum_{k=0}^{P} \binom{P}{k} 1^{P-k}\, 1^k = \sum_{k=0}^{P} \binom{P}{k}.$$

  Therefore, it follows that

  $$\mathrm{Card}(\mathcal{P}(E)) = (1+1)^P = 2^P.$$

3.2 Greedy Approach

- **Method:** Iterate through cells, pick the best local pair using a recursivity method, mark them used, and proceed until no more pairs can be formed. Here are the detailed mechanics:
  - First, it chooses a cell.
  - If the cell is black, it goes to the next.
  - Else, if the cell admit pairs, it computes the best pair associated.
    - It computes again the best pair with the cell already involved in the former pair.
    - If there is no second pair, it adds the first one to self.pairs()
    - If both pairs are the same, then there is a match. It adds it to self.pairs()
  - If not, it runs the algorithm again (recursivity) to find the best pair of the second cell, and again and again.
  - When one of the two first cases appears, the algorithm marks the cells forbidden to do not loop indefinitely.
- **Complexity:** In the worst case it is $\mathcal{O}((nm)^2)$, with some overhead for searching local neighbors. In facts the algorithm is pretty fast: **it solves grids of size 100×200 in less than one second**.
- **Proof:** Consider a grid of size $n \times m$, giving a total of nm cells. For the first cell, in the worst case, we may explore up to $(nm-1)$ other cells. For the second cell, at most $(nm-2)$ remain to be checked, and so on. Hence, for the k-th cell, there are at most $(nm-k)$ cells left to explore.

  Summing over all cells, we get:

$$\sum_{k=1}^{nm-1} k = 1 + 2 + 3 + \cdots + (nm-1) = \frac{(nm-1)(nm)}{2}.$$

  Since this sum is on the order of $(nm)^2$, the overall complexity is

$$O\big((nm)^2\big).$$

3.3 Ford-Fulkerson Algorithm

- **Method:** Build a flow network from the bipartite graph: a source connected to all even cells with capacity 1, odd cells to the sink also with capacity 1, and edges with capacity 1 for adjacent pairs. Then find the maximum matching using augmenting paths.
- **Complexity:** Typically $\mathcal{O}(|V||E|^2)$ for Edmonds-Karp. On a grid with nm cells, $|V| \approx nm$ and $|E|$ is proportional to valid adjacency edges.
- **Proof:** By adding each flow-augmenting path to the flow already established in the graph, the maximum flow is reached when no more augmenting paths can be found. However, there is no guarantee this situation will ever be reached in all cases, so the only certainty is that the solution is correct if the algorithm terminates. In pathological cases with irrational flow values, the algorithm may run forever without converging to the maximum flow.[1]

  When the capacities are *integer* values, the runtime of the Ford–Fulkerson algorithm is bounded by

$$O(E\,f),$$

[1] See, for instance, examples involving irrational capacities.

where E is the number of edges in the graph, and f is the maximum flow. This bound follows because each augmenting path can be found in $O(E)$ time, and each augmenting path increases the flow by at least 1 unit (due to integrality), up to the maximum flow f. Hence, the flow can be augmented at most f times.

3.4 Hungarian (Munkres) Method

- **Method:** Construct a bipartite graph that treats even-index cells as one set and odd-index cells as another, with edge costs set to $|\texttt{value}[A] - \texttt{value}[B]|$ if a pair is possible. Use Hungarian method to solve the assignment with the matrix method.
- **Complexity:** $\mathcal{O}((nm)^3)$ in the worst case. However, it is an exact polynomial-time solution for bipartite matching.
- **Proof:** Suppose there are J jobs and W workers with $J \leq W$. We describe how to compute, for each prefix of the jobs, the minimum total cost to assign those jobs to distinct workers. Eventually, this yields a matching that covers all J jobs. The total time to compute these matchings is

$$O\Big(\sum_{j=1}^{J} W\Big) = O(JW).$$

Note that this is better than $O\big(W^3\big)$ when $W \gg J$.

Adding the j-th Job in $O(W)$ Time. We maintain a *feasible* matching and adjust definitions as necessary. Let S_j be the set of the first j jobs, and let T be the set of all workers. A matching is feasible if each worker is assigned to at most one job. When the matching is *perfect* for S_j, all j jobs are matched to distinct workers.

During the j-th step:

- We add the job S_j to the previously considered set S_{j-1} and initialize a set $BZ = \varnothing$.
- At all times, every vertex in BZ belongs to a path starting from a job in some layer (Gy).
- While there exists a job or worker that is not matched, we run a BFS from that job (if it is currently unmatched).
- We define

$$\alpha \;=\; \min\big\{\, c(u,v) \;-\; Z(u) \;-\; Y(v) \;:\; (u,v) \in Z \cap S_j,\; v \in T \big\},$$

  and adjust the potentials accordingly (in the direction used in the previous section of the argument).
- After updating potentials, we repeat the BFS until either
  1. the new job is matched, or
  2. no augmenting path is found.

If there is no augmenting path in the subgraph of *tight edges* from u_0 to w_0, then toggling the first path found does not help, and the BFS ends when there is no free job. Otherwise, we use the path found to augment the matching, and then we update the potentials again.

Runtime Analysis.

- **Potential Updates:** Adjusting potentials takes $O(W^2)$ time in the worst case.
- **Recomputing D and w_{ext}:** After changing the potentials and Z, we can recompute D and w_{ext} in $O(W)$ time.
- **Case 1 Frequency:** The situation (Case 1 above) can occur at most once for each of the J steps. Even if it happens twice overall, the procedure terminates within $O(JW)$ time in total.

Hence, the overall time complexity for computing the assignments for all prefixes of jobs is $O(JW)$.

4 Experimental Results

We tested each solver on sample grids of dimensions 2×3, 4×8, 10×20, and so on.

Table 1: Representative scores on small and medium grids with **weighted values**.

Solver	**grid01.in (2x3)**	**grid05.in (4x8)**	**grid18.in (10x20)**
Greedy	6	41	291
Brute Force	6	None[1]	None[1]
Hungarian	6	35	273
Ford-Fulkerson	6	47	291

Table 2: Representative scores on small and medium grids with **no weighted values.**

Solver	**grid00.in (2x3)**	**grid05.in (4x8)**	**grid18.in (10x20)**
Greedy	1	5	37
Brute Force	1	None[1]	None[1]
Hungarian	1	3	29
Ford-Fulkerson	1	3	29

For each grid tested, all four solvers matched or closely approached the best-known value. The Greedy solution is fast and generally obtains an optimal or near-optimal result on these small instances. The Brute Force solver obviously obtains the optimal but becomes infeasible on large and even medium grids. Hungarian and Ford-Fulkerson methods are guaranteed to find the maximum matching with minimal cost, respectively for weighted edges and no weighted edges, making them both exact solutions for bipartite matching, albeit with polynomial complexity that can still be high for large grids.

5 Conclusion

In this report, we have summarized our project, which aims at pairing grid cells to minimize cost differences under specific adjacency and color constraints. We presented four main solver strategies, detailing both their complexity and performance on small to medium grids. Our extensive tests validate that the Brute Force approach achieves optimal solutions for small dimensions, while the Greedy approach is much faster and works fairly well in practice. The Hungarian and Ford-Fulkerson methods bring polynomial but higher theoretical complexity, yet they guarantee optimal solutions and scale decently up to moderate sizes.

[1]Too high complexity to be computed.

Overall, our final codebase is accompanied by unit tests covering `Grid` loading, adjacency-list graphs, and all solver implementations. These tests confirm that each algorithm performs as expected, consistently matching or closely matching known best scores.

6 Sources

- Wikipedia contributors. *Algorithme de Ford-Fulkerson.* `https://fr.wikipedia.org/wiki/Algorithme_de_Ford-Fulkerson`. Seen in January 2025.
- À la découverte des graphes. *Flots 2 : l'algorithme de Ford-Fulkerson pour construire un flot maximum dans un graphe - YouTube.* `https://www.youtube.com/watch?v=eL3fTl4mykY`. Seen in February 2025.
- Wikipedia contributors. *Algorithme hongrois.* `https://fr.wikipedia.org/wiki/Algorithme_hongrois`. Seen in March 2025.
- Wikipedia contributors. *Hungarian Algorithm.* `https://en.wikipedia.org/wiki/Hungarian_algorithm`. Seen in Februrary and March 2025.
- Nitish Korula and Abner Guzman-Rivera. *Maximum Weight Matching in Bipartite Graphs* `https://courses.grainger.illinois.edu/cs598csc/sp2010/lectures/lecture10.pdf`. Seen in February 2025.
- Thomas Dedecko. *Hungarian Algorithm (Python Implementation).* GitHub. `https://github.com/tdedecko/hungarian-algorithm`. Used in March 2025 for the project.

Keywords: grid pairing, bipartite matching, complexity, adjacency lists, cost minimization.